

Math 707 – Final Project

Interior-Point Methods for Constrained Portfolio Optimization

Saeed Alameri

100065780

Math 707 – Nonlinear Optimization

Spring 2026

Project: “Interior-Point Methods for Constrained Portfolio Optimization”

Abstract

This project studies the logarithmic barrier (interior-point) method applied to Markowitz mean-variance portfolio optimization with no-short-selling constraints. The problem is formulated as a convex quadratic program: minimize portfolio variance subject to a target return, a budget constraint, and non-negativity of all weights. The barrier method replaces the inequality constraints with a logarithmic penalty and solves a sequence of equality-constrained subproblems by Newton’s method. The main convergence properties are stated: the suboptimality bound decreases as $O(1/t)$ in the barrier parameter, while the inner Newton iterations converge quadratically. The key assumptions are convexity of the objective, Slater’s condition for the constraints, and Lipschitz continuity of the Hessian. The method is implemented in Python on a simulated 5-asset problem and on a real-data portfolio built from daily returns of Apple, Alphabet, Meta, Amazon, and Microsoft over the fixed window 2023-01-03 to 2025-12-30, and is compared against projected gradient descent and against the primal-dual interior-point variant. On the simulated problem the barrier method reaches high accuracy in 38 Newton steps in 1.5 ms, the primal-dual variant in 12 iterations in 0.4 ms, while projected gradient descent needs 175 iterations and about 91 ms for comparable accuracy. The same ranking is observed on the real-data problem.

Contents

1	Introduction and Motivation	3
2	Problem Formulation	3
2.1	Mathematical Model	4
2.2	Assumptions and Structure	4
3	Methodology: Algorithm Description	5
3.1	Baseline Method	5
3.2	Main Algorithm	6
3.3	Pseudocode	6
3.4	Variant: Primal-Dual Interior-Point Method	7
4	Theoretical Properties	7
4.1	Convergence Statement	8
4.2	Discussion of Assumptions	8

5	Numerical Experiment	9
5.1	Experimental Setup	9
5.2	Results	10
5.3	Real-Data Experiment	11
6	Discussion	12
7	Conclusion and Outlook	12
A	Implementation Code	13

1 Introduction and Motivation

Portfolio optimization is one of the most well-known applications of mathematical optimization. The basic question is: given a set of assets, how should an investor distribute their wealth across these assets so that risk is minimized for a desired level of return? This question was first studied by Markowitz [1], who introduced the mean-variance framework. In this framework, the expected return of a portfolio is modeled using the mean of asset returns, and the risk is modeled using the variance, which depends on the covariance matrix of the returns.

In many practical situations, the investor is not allowed to take short positions. Short-selling means selling an asset that one does not currently own, with the hope of buying it back later at a lower price. When short-selling is not allowed, all portfolio weights must be non-negative. This introduces inequality constraints into the optimization problem, turning it into a constrained quadratic program.

A natural and well-studied approach to handle inequality constraints is the **interior-point method** (also called the barrier method). The main idea is to replace the hard inequality constraints with a smooth penalty term, called a **logarithmic barrier function**. This barrier function takes very large values as the iterates approach the boundary of the feasible region. As a result, the optimization algorithm stays in the interior of the feasible set, and the constraints are never violated during the iterations. At each step, the method uses first and second derivatives (gradient and Hessian) to find a descent direction, similar to Newton's method. A parameter t , called the barrier parameter, controls how closely the barrier problem approximates the original constrained problem. As t increases across outer iterations, the solution of the barrier subproblem converges to the solution of the original problem.

This approach is attractive because it converts a constrained problem into a sequence of unconstrained (or equality-constrained) problems, each of which can be solved efficiently using Newton steps. The theoretical convergence properties of interior-point methods are also well understood: under mild assumptions such as convexity and the existence of a strictly feasible point, the method converges at a rate of $O(1/t)$ in terms of the suboptimality gap [2].

The goal of this project is to study the interior-point method in the context of portfolio optimization with no-short-selling constraints. We formulate the problem as a convex quadratic program, describe the barrier method, state its convergence properties, and implement a numerical example.

The main references for this project are the original work of Markowitz [1] on portfolio selection, the textbook by Boyd and Vandenberghe [2] on convex optimization and barrier methods, and the textbook by Wright [3] which provides a detailed treatment of primal-dual interior-point methods.

2 Problem Formulation

We consider the classical Markowitz mean-variance portfolio optimization problem. Suppose an investor has access to n assets. Let $w \in \mathbb{R}^n$ be the vector of portfolio weights, where w_i represents the fraction of the total wealth invested in asset i . The investor wants to minimize the portfolio risk (measured by variance) while achieving at least a target expected return.

Let $\mu \in \mathbb{R}^n$ be the vector of expected returns of the n assets, and let $\Sigma \in \mathbb{R}^{n \times n}$ be the covariance matrix of the asset returns. We assume that Σ is symmetric and positive semidefinite, which is a standard assumption in portfolio theory. In most practical cases, Σ is in fact positive definite, meaning no asset return can be written as an exact linear combination of the others.

The portfolio return is $\mu^\top w$ and the portfolio variance is $w^\top \Sigma w$. The investor requires the following:

- The portfolio weights must sum to one: $\mathbf{1}^\top w = 1$. This means all wealth is invested.
- Each weight must be non-negative: $w_i \geq 0$ for all $i = 1, \dots, n$. This is the no-short-selling constraint.
- The expected return must be at least a target value $r_{\min}$: $\mu^\top w \geq r_{\min}$.

Notation. The decision variable is the portfolio weight vector $w \in \mathbb{R}^n$, where w_i is the fraction of total wealth invested in asset i . The return data are the expected return vector $\boldsymbol{\mu} \in \mathbb{R}^n$ and the covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$, both evaluated on the same return horizon. In the real-data experiment of Section 5.3, $\boldsymbol{\mu}$ and Σ are annualized from daily simple returns by the standard scaling factor 252. The vector of ones is denoted by $\mathbf{1}$, the target return by $r_{\min}$, and the return slack by $s = \boldsymbol{\mu}^\top w - r_{\min}$. For the barrier method, $t > 0$ is the barrier parameter, $\gamma > 1$ the multiplicative growth factor between outer iterations, and $\epsilon > 0$ the duality-gap tolerance. For the primal-dual method, $\nu \in \mathbb{R}$ denotes the multiplier for the budget equality, $\eta \geq 0$ the multiplier for the return inequality, $\lambda \in \mathbb{R}_{\geq 0}^n$ the multipliers for the non-negativity inequalities, and $\tau > 0$ the relaxation parameter for complementarity; the associated duality measure d_k is defined in Section 3.4. The factor $\frac{1}{2}$ in the objective $f(w) = \frac{1}{2} w^\top \Sigma w$ is included for algebraic convenience, since it cancels in $\nabla f(w) = \Sigma w$, and does not change the optimizer of (1).

2.1 Mathematical Model

Using the notation above, the optimization problem is:

$$\begin{aligned} \min_{w \in \mathbb{R}^n} \quad & \frac{1}{2} w^\top \Sigma w \\ \text{subject to} \quad & \boldsymbol{\mu}^\top w \geq r_{\min}, \\ & \mathbf{1}^\top w = 1, \\ & w_i \geq 0, \quad i = 1, \dots, n. \end{aligned} \tag{1}$$

This is a convex quadratic program. The objective function $f(w) = \frac{1}{2} w^\top \Sigma w$ is convex (and strictly convex when Σ is positive definite). All constraints are linear, so the feasible set is a convex polytope. Therefore, any local minimizer is also a global minimizer.

The problem has one equality constraint ($\mathbf{1}^\top w = 1$), one linear inequality for the return target ($\boldsymbol{\mu}^\top w \geq r_{\min}$), and n non-negativity constraints ($w_i \geq 0$). In total, there are $n + 1$ inequality constraints and one equality constraint. The decision variable w lives in $\mathbb{R}^n$.

To apply the interior-point method, we will incorporate the inequality constraints into a logarithmic barrier function and keep the equality constraint as an explicit constraint in the barrier subproblem. This is described in the Methodology section.

2.2 Assumptions and Structure

We now list the structural assumptions that we use throughout the project and explain how they connect to the theory covered in the course.

Assumption 1: Positive definiteness of Σ . We assume the covariance matrix Σ is symmetric positive definite, that is, $\Sigma \succ 0$. This means $w^\top \Sigma w > 0$ for every nonzero $w \in \mathbb{R}^n$. As a consequence, the objective function $f(w) = \frac{1}{2} w^\top \Sigma w$ is *strictly convex*. In fact, it is also *strongly convex* with parameter $m = \lambda_{\min}(\Sigma) > 0$, where $\lambda_{\min}(\Sigma)$ denotes the smallest eigenvalue of Σ . Strong convexity guarantees that the minimizer of (1) is unique (see Chapter 3, Proposition 9 in [6]).

In practice, positive definiteness holds when no asset return is a perfect linear combination of the others. If the number of assets n is larger than the number of historical return observations used to estimate Σ , then Σ may only be positive semidefinite. In that case, one common fix is to add a small regularization term ϵI to Σ , which restores positive definiteness.

Assumption 2: Linear constraints. All constraints in (1) are linear (or affine) in the decision variable w . Specifically:

- The budget constraint $\mathbf{1}^\top w = 1$ is an affine equality.

- The return constraint $\boldsymbol{\mu}^\top w \geq r_{\min}$ is a linear inequality.
- The non-negativity constraints $w_i \geq 0$ are linear inequalities.

Because the objective is convex and all constraints are linear, the feasible set is a convex polytope (intersection of halfspaces and a hyperplane). The problem is therefore a *convex quadratic program*. From the theory of convex optimization, this means that the KKT conditions are both necessary and sufficient for global optimality (Chapter 5, Theorem 17 in [6]).

Assumption 3: Slater’s condition. For interior-point methods to work properly, we need a constraint qualification to hold. The most natural one for convex problems is **Slater’s condition** (Chapter 8, Definition 42 in [6]). Slater’s condition requires the existence of a *strictly feasible point*, meaning a point $\bar{w}$ that satisfies all inequality constraints with strict inequality:

$$\bar{w}_i > 0 \text{ for all } i, \quad \boldsymbol{\mu}^\top \bar{w} > r_{\min}, \quad \mathbf{1}^\top \bar{w} = 1.$$

For the portfolio problem considered here, Slater’s condition is verified directly by the uniform portfolio $\bar{w} = (1/n, \dots, 1/n)^\top$, which satisfies $\bar{w}_i = 1/n > 0$ and $\mathbf{1}^\top \bar{w} = 1$. Whenever the target return $r_{\min}$ is chosen below the uniform-portfolio return $\boldsymbol{\mu}^\top \bar{w}$, this point is strictly feasible.

Slater’s condition has two important consequences:

1. **Strong duality** holds (Chapter 6, Theorem 19 in [6]): the duality gap is zero, and dual optimal solutions exist.
2. **The central path exists:** the barrier subproblems have well-defined solutions for all values of the barrier parameter, and these solutions converge to the optimal solution of the original problem as the barrier parameter $t \rightarrow \infty$ [4].

Summary of problem structure. To summarize, problem (1) is a convex quadratic program with a strictly convex objective, linear constraints, and a strictly feasible interior. This is a well-behaved setting for the interior-point method. The key theoretical tools from the course [6] that we rely on are: convexity and strong convexity (Chapter 3), KKT conditions and their sufficiency for convex problems (Chapter 5), strong duality (Chapter 6), Slater’s condition (Chapter 8), and the barrier method and central path (Chapter 14).

3 Methodology: Algorithm Description

We now describe the barrier method applied to the portfolio problem (1). The idea, following Chapter 14 of [6] and Chapter 11 of [2], is to replace the inequality constraints with a logarithmic barrier term and solve a sequence of equality-constrained subproblems using Newton’s method.

3.1 Baseline Method

As a baseline for comparison, we use **projected gradient descent**. At each iteration, we take a step in the negative gradient direction and then project the result back onto the feasible set:

$$w^{(k+1)} = \Pi_{\mathcal{C}}\left(w^{(k)} - \alpha \Sigma w^{(k)}\right),$$

where $\alpha > 0$ is the step size and $\Pi_{\mathcal{C}}$ denotes the Euclidean projection onto the feasible set $\mathcal{C} = \{w : \mathbf{1}^\top w = 1, w \geq 0, \boldsymbol{\mu}^\top w \geq r_{\min}\}$. The gradient of the objective is $\nabla f(w) = \Sigma w$.

Projected gradient descent is a first-order method. It only uses gradient information and does not use curvature (Hessian) information. Its convergence rate is linear, and it can be slow when the condition number of Σ is large (Chapter 9, Remark 50 in [6]). We include it as a reference point to show the advantage of using second-order information in the interior-point method.

3.2 Main Algorithm

Step 1: Logarithmic barrier function. We rewrite the $n + 1$ inequality constraints in standard form $g_i(w) \leq 0$: the non-negativity constraints become $g_i(w) = -w_i \leq 0$ for $i = 1, \dots, n$, and the return constraint becomes $g_{n+1}(w) = r_{\min} - \boldsymbol{\mu}^\top w \leq 0$. The logarithmic barrier function is

$$\phi(w) = - \sum_{i=1}^n \log(w_i) - \log(\boldsymbol{\mu}^\top w - r_{\min}). \quad (2)$$

This function is defined only for strictly feasible points ($w_i > 0$ and $\boldsymbol{\mu}^\top w > r_{\min}$) and goes to $+\infty$ as w approaches the boundary of the feasible region.

Step 2: Barrier subproblem. For a given barrier parameter $t > 0$, we solve the subproblem

$$\min_w \underbrace{t \cdot \frac{1}{2} w^\top \Sigma w}_{\text{scaled variance}} + \underbrace{\phi(w)}_{\text{log-barrier term}} \quad \text{subject to} \quad \underbrace{\mathbf{1}^\top w = 1}_{\text{budget constraint}}. \quad (3)$$

As t increases, the barrier term $\phi(w)$ becomes relatively less important, and the solution of (3) approaches the solution of (1). The set of solutions $\{w^*(t)\}_{t>0}$ is called the *central path* [2].

Step 3: Newton's method for the subproblem. Each subproblem (3) is a smooth, strictly convex optimization problem with one equality constraint. We solve it using Newton's method. Let $s = \boldsymbol{\mu}^\top w - r_{\min}$ denote the return slack. The gradient and Hessian of the objective in (3) are:

$$g(w) = t \Sigma w - \begin{pmatrix} 1/w_1 \\ \vdots \\ 1/w_n \end{pmatrix} - \frac{\boldsymbol{\mu}}{s}, \quad (4)$$

$$H(w) = t \Sigma + \text{diag}\left(\frac{1}{w_1^2}, \dots, \frac{1}{w_n^2}\right) + \frac{\boldsymbol{\mu} \boldsymbol{\mu}^\top}{s^2}. \quad (5)$$

The diagonal and rank-one terms in the Hessian come from the second derivatives of the logarithmic barrier (2). Note that $H(w)$ is always positive definite for strictly feasible w , since all three terms are positive semidefinite and $t \Sigma \succ 0$.

To find the Newton step Δw while respecting the equality constraint $\mathbf{1}^\top w = 1$, we solve the KKT system of the Newton subproblem [4]:

$$\underbrace{\begin{pmatrix} H & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix}}_{\text{KKT matrix}} \underbrace{\begin{pmatrix} \Delta w \\ \nu \end{pmatrix}}_{\text{Newton step and multiplier}} = \underbrace{\begin{pmatrix} -g \\ 0 \end{pmatrix}}_{\text{negative gradient, budget preserved}}, \quad (6)$$

where $\nu \in \mathbb{R}$ is the dual variable for the equality constraint. The right-hand side has a zero in the second block because the current iterate already satisfies $\mathbf{1}^\top w = 1$, so we need $\mathbf{1}^\top \Delta w = 0$.

Step 4: Outer loop. We start with a moderate value of t and multiply it by a factor $\gamma > 1$ after each subproblem is solved. The outer loop stops when the duality gap estimate $(n + 1)/t$ is below a tolerance $\epsilon > 0$. The quantity $(n + 1)/t$ is a known upper bound on the suboptimality $f(w^*(t)) - f(w^*)$ [2].

3.3 Pseudocode

Algorithm: Barrier Method for Portfolio Optimization

Input: Strictly feasible initial point $w^{(0)}$, initial barrier parameter $t_0 > 0$, growth factor $\gamma > 1$, outer-loop tolerance $\epsilon > 0$, Armijo constant $c_1 \in (0, 1)$, inner tolerance $\eta_N > 0$.

Let $F_t(w) = t \cdot \frac{1}{2} w^\top \Sigma w + \phi(w)$ denote the barrier objective in (3).

For $k = 0, 1, 2, \dots$ until $(n+1)/t_k < \epsilon$:

1. **Centering step.** Starting from $w^{(k)}$, repeat the following inner Newton iteration until $\|\alpha \Delta w\| < \eta_N$:

- Compute gradient $g(w)$ and Hessian $H(w)$ via (4)–(5).
- Solve the KKT system (6) for the Newton direction Δw .
- *Backtracking line search.* Set $\alpha \leftarrow 1$ and halve α until both
 - (a) strict feasibility holds: $w + \alpha \Delta w > 0$ componentwise and $\mu^\top(w + \alpha \Delta w) > r_{\min}$;
 - (b) sufficient decrease holds: $F_{t_k}(w + \alpha \Delta w) \leq F_{t_k}(w) + c_1 \alpha g(w)^\top \Delta w$.
- Apply the damped update $w \leftarrow w + \alpha \Delta w$.

Set $w^{(k+1)}$ to the result of the centering step.

2. **Increase barrier parameter.** Set $t_{k+1} = \gamma \cdot t_k$.

Output: Approximate solution $w^* \approx w^{(k+1)}$.

3.4 Variant: Primal-Dual Interior-Point Method

As a second reference method, the primal-dual interior-point variant [3] is also implemented. This variant is the algorithm used by modern conic and quadratic solvers such as MOSEK and Gurobi. Instead of solving each barrier subproblem to completion before increasing t , the primal-dual method works with the primal variable w and the dual variables (ν, η, λ) simultaneously, where $\eta \geq 0$ is the multiplier of the return inequality and $\lambda \in \mathbb{R}_{\geq 0}^n$ are the multipliers of the non-negativity inequalities. The exact complementarity conditions $\eta(\mu^\top w - r_{\min}) = 0$ and $\lambda_i w_i = 0$ are replaced by their relaxed counterparts

$$\underbrace{\eta \cdot (\mu^\top w - r_{\min})}_{\text{return-slack complementarity}} = \underbrace{\tau}_{\text{relaxation}}, \quad \underbrace{\lambda_i \cdot w_i}_{\text{non-negativity complementarity}} = \underbrace{\tau_i}_{\text{relaxation}}, \quad i = 1, \dots, n, \quad (7)$$

which are driven to zero along a single sequence of Newton steps applied to the full perturbed KKT system. The relaxation parameter is updated as $\tau = \sigma d_k$, where $\sigma \in (0, 1)$ is a centering parameter and the duality measure is defined by

$$d_k = \frac{1}{n+1} (\eta(\mu^\top w - r_{\min}) + w^\top \lambda). \quad (8)$$

The implementation takes one Newton step per iteration on the perturbed KKT system, applies a fraction-to-boundary rule that keeps w , λ , η , and the return slack s strictly positive, and terminates when d_k falls below the prescribed tolerance. The variant is included only as an additional benchmark; it does not constitute a new theoretical contribution.

4 Theoretical Properties

This section states the main convergence results that apply to the barrier method described above. We do not give full proofs, but we state the results carefully and explain what the assumptions mean in the present setting. The presentation follows Chapter 11 of [2] and Chapter 14 of [6].

4.1 Convergence Statement

The barrier method has two levels: an outer loop that increases t , and an inner loop that solves each barrier subproblem using Newton's method. We state the convergence properties of each level separately.

Outer loop: suboptimality bound. Let $w^*(t)$ denote the solution of the barrier subproblem (3) for a given t , and let w^* denote the solution of the original problem (1). Then the following bound holds [2]:

$$f(w^*(t)) - f(w^*) \leq \frac{n+1}{t}, \quad (9)$$

where $n+1$ is the total number of inequality constraints (n non-negativity constraints plus one return constraint). This result assumes that the objective f is convex, the constraints are convex, and Slater's condition holds.

Since we multiply t by $\gamma > 1$ at each outer iteration, after k outer iterations we have $t_k = \gamma^k t_0$. Therefore the suboptimality after k outer steps is at most $(n+1)/(\gamma^k t_0)$. To reach a suboptimality of at most ϵ , we need

$$k \geq \frac{\log((n+1)/(t_0 \epsilon))}{\log \gamma}$$

outer iterations. This means the number of outer iterations grows only *logarithmically* in $1/\epsilon$, which is very efficient.

Inner loop: quadratic convergence of Newton's method. Each barrier subproblem is solved by Newton's method. Since the barrier objective $t f(w) + \phi(w)$ is twice continuously differentiable and strictly convex on the strictly feasible set, and since the Hessian $H(w)$ is positive definite, Newton's method converges *quadratically* once the iterates are close enough to the minimizer $w^*(t)$. More precisely (Chapter 10, Theorem 33 in [6]): if $\nabla^2 f$ is Lipschitz continuous near $w^*(t)$ and w is sufficiently close to $w^*(t)$, then

$$\|w^+ - w^*(t)\| \leq C \|w - w^*(t)\|^2$$

for some constant $C > 0$, where w^+ is the next Newton iterate. Quadratic convergence means that the number of correct digits roughly doubles at each Newton step. In practice, this means that only a small number of Newton iterations (often 5 to 10) is needed to solve each barrier subproblem to high accuracy. In the experiment of Section 5, each centering step required between 2 and 5 Newton iterations.

Overall convergence. Combining both levels, the barrier method converges to the global minimizer w^* of problem (1). The total cost is $O(\log(1/\epsilon))$ outer iterations, each requiring a small number of Newton steps. This is much faster than projected gradient descent, which has a linear convergence rate that depends on the condition number of the problem [5].

Connection to KKT conditions. As $t \rightarrow \infty$, the central path solutions $w^*(t)$ converge to a point that satisfies the KKT conditions of the original problem (1) (Chapter 5, Theorem 16 in [6]). The dual variables can be recovered from the barrier solution: for each inequality constraint $g_i(w) \leq 0$, the corresponding approximate Lagrange multiplier is $\lambda_i \approx -1/(t g_i(w^*(t)))$. As t grows, these approximate multipliers converge to the true KKT multipliers [3].

4.2 Discussion of Assumptions

This subsection discusses whether the assumptions required for the convergence results above are satisfied in the portfolio problem at hand.

Convexity. The objective $f(w) = \frac{1}{2} w^\top \Sigma w$ is convex whenever $\Sigma \succeq 0$, and strictly convex when $\Sigma \succ 0$. In our numerical example we use a positive definite covariance matrix, so this assumption is satisfied. If Σ were only positive semidefinite, the minimizer would still exist (the feasible set is compact), but it might not be unique.

Slater’s condition. As discussed in Section 2.2, Slater’s condition holds for the present problem whenever the target return $r_{\min}$ is not too large. The experiment selects $r_{\min}$ so that the uniform portfolio is strictly feasible, which verifies the assumption. If $r_{\min}$ is set equal to the maximum possible return (the return of the highest-yielding asset), then Slater’s condition can fail because the strictly feasible set may become empty. In that boundary case, the central path is not guaranteed to exist, and the barrier method may not converge properly.

Smoothness of the Hessian. The quadratic convergence of Newton’s method requires the Hessian of the barrier objective to be Lipschitz continuous near the solution. For the present problem, the Hessian (5) involves terms $1/w_i^2$ and $1/s^2$. These are smooth as long as w_i and $s = \boldsymbol{\mu}^\top w - r_{\min}$ stay bounded away from zero. Along the central path, strict feasibility is maintained, so these terms remain well-behaved. However, as $t \rightarrow \infty$, some components $w_i^*(t)$ may approach zero (if the corresponding asset is not in the optimal portfolio). This causes the Hessian to become increasingly ill-conditioned for large t , which is a known difficulty of barrier methods [4]. In practice, this is managed by not letting t grow too large and by using the warm-start from the previous subproblem.

Summary. All three assumptions (convexity, Slater’s condition, Hessian smoothness) are satisfied in the numerical example studied here. The barrier method is therefore guaranteed to converge to the global minimizer of the portfolio problem, and the convergence is fast: logarithmic in $1/\epsilon$ for the outer loop and quadratic for the inner Newton steps.

5 Numerical Experiment

The barrier method described in Section 3 is implemented on a small portfolio optimization problem with $n = 5$ assets. The implementation is in Python and uses NumPy for linear algebra. The barrier method is compared against projected gradient descent (baseline) and against the primal-dual variant of Section 3.4 (additional reference). All reported timings are wall-clock seconds on a MacBook Air 13-inch (Apple M3, 2024); since each algorithm is deterministic, the median over five repeated runs is reported to reduce timing noise rather than to characterize algorithmic variability.

5.1 Experimental Setup

Problem data. We consider $n = 5$ assets with the following expected returns:

$$\boldsymbol{\mu} = (0.12, 0.10, 0.07, 0.03, 0.05)^\top.$$

The covariance matrix is constructed as follows. We first generate a random 5×5 matrix A with entries drawn from a standard normal distribution (using a fixed random seed for reproducibility), and then set $\Sigma = A^\top A / 100 + 0.01 I$ to ensure positive definiteness. The resulting Σ has condition number approximately 13.6.

The target return is set to $r_{\min} = 0.06$, which is below the average return $\boldsymbol{\mu}^\top \bar{w} = 0.074$ of the uniform portfolio $\bar{w} = (0.2, 0.2, 0.2, 0.2, 0.2)^\top$. This ensures that Slater’s condition is satisfied.

Algorithm parameters. For the barrier method:

- Initial point: $w^{(0)} = (0.2, 0.2, 0.2, 0.2, 0.2)^\top$ (uniform portfolio).
- Initial barrier parameter: $t_0 = 1$.

- Growth factor: $\gamma = 10$.
- Outer loop tolerance: $\epsilon = 10^{-8}$ (stop when $(n+1)/t < \epsilon$).
- Inner Newton tolerance: stop when $\|\Delta w\| < 10^{-10}$.

For projected gradient descent:

- Initial point: same $w^{(0)}$.
- Step size: $\alpha = 1/\lambda_{\max}(\Sigma)$ (the reciprocal of the largest eigenvalue of Σ , which guarantees convergence).
- Maximum iterations: 10,000.
- Tolerance: stop when $\|w^{(k+1)} - w^{(k)}\| < 10^{-8}$.

For the primal-dual variant:

- Centering parameter $\sigma = 0.1$ and fraction-to-boundary safety factor 0.995.
- Initial multipliers $\eta = 1$, $\lambda = \mathbf{1}$, $\nu = 0$, and primal $w^{(0)}$ as above.
- Tolerance: stop when the duality measure d_k defined in (8) falls below 10^{-10} .

5.2 Results

Barrier method convergence. Table 1 shows the progress of the barrier method across outer iterations. The duality gap $(n+1)/t$ decreases by a factor of 10 at each outer step (since $\gamma = 10$), and the objective value converges rapidly. Each centering step requires only 2 to 5 Newton iterations to reach high accuracy.

Outer iter. k	t_k	Newton steps	Objective $f(w^{(k)})$	Gap $(n+1)/t_k$
0	1	5	8.054×10^{-3}	6.00
1	10	4	7.977×10^{-3}	6.00×10^{-1}
2	10^2	5	7.345×10^{-3}	6.00×10^{-2}
3	10^3	5	5.677×10^{-3}	6.00×10^{-3}
4	10^4	5	5.328×10^{-3}	6.00×10^{-4}
5	10^5	4	5.303×10^{-3}	6.00×10^{-5}
6	10^6	3	5.302×10^{-3}	6.00×10^{-6}
7	10^7	3	5.302×10^{-3}	6.00×10^{-7}
8	10^8	2	5.302×10^{-3}	6.00×10^{-8}
9	10^9	2	5.302×10^{-3}	6.00×10^{-9}

Table 1: Convergence of the barrier method. The objective stabilizes after a few outer iterations, and the duality gap decreases geometrically.

Comparison against the baseline and the variant. Table 2 compares the three methods on the simulated problem. The barrier method reaches the same accuracy in far fewer total gradient and Hessian evaluations than projected gradient descent, which reflects the quadratic convergence of Newton’s method in the inner loop. The primal-dual variant requires the fewest iterations because it takes a single Newton step on the perturbed KKT system per iteration instead of solving an inner subproblem at each centering step. The wall-clock times follow the same order. Projected gradient descent carries a substantial per-iteration cost because every step performs a nested SLSQP projection onto the feasible set.

Method	Iterations	Time (s)	$f(w^{(k)}) = \frac{1}{2}w^\top \Sigma w$	$\ w^{(k)} - w^*\ $
Projected gradient descent	175	0.0905	5.302×10^{-3}	1.44×10^{-6}
Barrier method	38	0.0015	5.302×10^{-3}	$< 10^{-9}$
Primal-dual IPM	12	0.0004	5.302×10^{-3}	1.54×10^{-6}

Table 2: Comparison of the three methods on the simulated 5-asset problem. The “Iterations” column reports outer projected-gradient steps for the baseline, total inner Newton steps (summed across all centering steps) for the barrier method, and combined primal-dual steps on the perturbed KKT system for the variant. The reported objective is the half-variance $f(w) = \frac{1}{2}w^\top \Sigma w$. Times are median wall-clock seconds over five repeated runs on a MacBook Air 13-inch (Apple M3, 2024).

Optimal portfolio. The computed optimal portfolio weights are approximately:

$$w^* \approx (0.251, 0.041, 0.526, 0.042, 0.139)^\top.$$

Asset 3 receives the largest weight (0.526), even though asset 1 has the highest expected return (0.12). This is because asset 3 has the lowest variance on the diagonal of Σ (approximately 0.022, compared to 0.040 for asset 1). The optimizer favors asset 3 for its better risk-to-return balance. Assets 2 and 4 receive small weights (around 0.04 each) because their risk-return profiles are less favorable given the other available assets. Overall, the solution balances diversification and return achievement in a way that is consistent with mean-variance theory.

5.3 Real-Data Experiment

To assess whether the ranking observed on the simulated data persists on non-synthetic inputs, the experiment is repeated on a small real-data instance. Daily adjusted closing prices for Apple (AAPL), Alphabet (GOOGL), Meta (META), Amazon (AMZN), and Microsoft (MSFT) are obtained over the fixed window from 2023-01-03 to 2025-12-30 (751 trading days) using the `yfinance` package. Daily simple returns are annualized by the factor 252, yielding the mean vector

$$\mu \approx (0.300, 0.472, 0.636, 0.386, 0.274)^\top$$

and an empirical covariance matrix Σ with condition number approximately 11.92. The target return is set to one half of the largest mean return, $r_{\min} \approx 0.318$, which lies below the uniform-portfolio return of approximately 0.413 and therefore preserves Slater’s condition. Table 3 reports the iteration count, the median wall-clock time, and the final half-variance objective for each method. The ranking matches that of the simulated problem: the primal-dual variant attains the smallest wall-clock time, the barrier method is second, and projected gradient descent is the slowest by approximately one order of magnitude. The computed optimal portfolio places weight 0.484 on MSFT, 0.336 on AAPL, and 0.180 on GOOGL, while the weights assigned to META and AMZN are at the level of the SLSQP projection tolerance ($\sim 10^{-6}$), which is consistent with the boundary multipliers identified by the primal-dual variant.

Method	Iterations	Time (s)	$f(w^{(k)}) = \frac{1}{2}w^\top \Sigma w$	$\ w^{(k)} - w^*\ $
Projected gradient descent	129	0.0921	2.142×10^{-2}	3.95×10^{-6}
Barrier method	68	0.0039	2.142×10^{-2}	$< 10^{-6}$
Primal-dual IPM	13	0.0005	2.142×10^{-2}	3.88×10^{-6}

Table 3: Comparison on the real-data instance built from daily returns of AAPL, GOOGL, META, AMZN, and MSFT over 2023-01-03 to 2025-12-30. Iteration counts are interpreted as in Table 2. The reported objective is the half-variance $f(w) = \frac{1}{2}w^\top \Sigma w$ on the annualized covariance matrix.

6 Discussion

The numerical results confirm the theoretical predictions from Section 4.

Convergence behavior. The barrier method converged as predicted by the theory. The duality gap decreased by a factor of $\gamma = 10$ at each outer iteration, consistent with the bound (9). Each centering step terminated within 2 to 5 inner Newton iterations, with fewer steps required in later outer iterations because the warm-start lies close to the new central-path point. This behavior is characteristic of quadratic convergence and matches the analysis in Chapter 10 of [6]. Projected gradient descent also converged to the correct solution, but it required 175 iterations compared to 38 total Newton steps for the barrier method.

Sensitivity to parameters. The growth factor γ was varied to assess its effect. With $\gamma = 2$, the outer loop required 31 iterations (instead of 10) to reach the same tolerance, with typically 4 Newton steps per centering and 108 inner Newton steps in total. With $\gamma = 100$, the outer loop required only 6 iterations, with 3 to 5 Newton steps per centering and 24 Newton steps in total. The intermediate choice $\gamma = 10$ (38 total Newton steps) is consistent with the recommendation in [2] and lies near the empirical minimum of total work for this instance.

The initial barrier parameter t_0 also affects the trade-off. A small t_0 makes the first barrier subproblems easier to solve, since the barrier term dominates, but increases the number of outer iterations. A large t_0 reduces the outer iterations, but the first centering step can be difficult when the initial point is far from the central path.

Strengths of the barrier method. The dominant advantage measured here is convergence speed. Because each Newton step uses both gradient and Hessian information, the barrier method attains a high-accuracy solution in a small number of total iterations. The method also keeps every iterate strictly feasible, with positive weights and a positive return slack, which is a useful structural property in practice.

Limitations. The main cost per iteration is solving the $(n + 1) \times (n + 1)$ linear system (6). For the small example with $n = 5$, this is inexpensive. For large-scale problems with hundreds or thousands of assets, the cost of forming and factorizing the Hessian becomes significant. In such cases, one would need to exploit the structure of Σ (for example, sparsity or factor-model structure) to make the linear solves efficient. Another limitation is that the barrier method requires a strictly feasible starting point, which may not always be easy to find for more complex constraint sets.

7 Conclusion and Outlook

This project studied the logarithmic barrier (interior-point) method for Markowitz portfolio optimization with no-short-selling constraints. The problem was formulated as a convex quadratic program, the barrier subproblem and its Newton system were derived, and convergence was analyzed using tools from the course [6]: convexity, KKT conditions, Slater’s condition, and the central path. The numerical experiment on a 5-asset simulated problem confirmed that the barrier method converges substantially faster than projected gradient descent, in agreement with theory. The primal-dual interior-point variant [3] was also implemented, and the three methods were compared on a real-data portfolio built from daily returns of Apple, Alphabet, Meta, Amazon, and Microsoft over the fixed window 2023-01-03 to 2025-12-30. The primal-dual variant attained the smallest wall-clock time, the barrier method was second, and projected gradient descent was the slowest by approximately one order of magnitude on both instances. Possible extensions include scaling to portfolios with hundreds of assets by exploiting sparsity or factor structure in Σ , and incorporating additional constraints such as sector limits or transaction costs.

A Implementation Code

The following Python script implements all three methods compared in Section 5: the barrier method, the primal-dual interior-point variant, and projected gradient descent, together with the simulated and real-data problem builders. The script requires NumPy, SciPy, and `yfinance` (the last only for the real-data experiment). The optional `curl_cffi` package is used as a Yahoo Finance session when available, in order to avoid rate limiting. All results in Section 5 can be reproduced by running `python3 experiment.py`, which also writes a machine-readable `results.json` file with iteration counts, timings, and final weights for both experiments.

Listing 1: All three methods, the simulated 5-asset problem, and the real-data problem (`experiment.py`)

```
1 """Interior-point methods for Markowitz portfolio optimization.
2
3 This script reproduces the numerical results reported in the project. Three
4 methods are applied to the same constrained quadratic program:
5
6     1. Projected gradient descent (first-order baseline).
7     2. Logarithmic barrier method (sequential centering with Newton's method).
8     3. Primal-dual interior-point method (single Newton step on the relaxed
9       KKT system).
10
11 Two experiments are run:
12
13     A. A simulated 5-asset problem with a fixed random seed.
14     B. A real-data problem built from daily adjusted closing prices of
15       Apple (AAPL), Alphabet (GOOGL), Meta (META), Amazon (AMZN), and
16       Microsoft (MSFT) over the fixed window 2023-01-03 to 2025-12-31.
17
18 Reported wall-clock times were measured on a MacBook Air 13-inch (Apple M3,
19 2024). For each method the median over five repeated runs is reported in
20 order to reduce timing noise.
21
22 Dependencies:
23     NumPy, SciPy. The real-data experiment also requires yfinance; the
24     optional curl_cffi package is used to avoid Yahoo rate-limiting.
25
26 Run:
27     python3 experiment.py
28 """
29
30 import json
31 import time
32 from dataclasses import dataclass
33 from pathlib import Path
34 from typing import Callable, Sequence
35
36 import numpy as np
37 from scipy.optimize import minimize as sp_minimize
38
39
40 # Fixed window used by the real-data experiment.
41 REAL_DATA_START = "2023-01-03"
42 REAL_DATA_END = "2025-12-31"
43 TRADING_DAYS_PER_YEAR = 252
44 RESULTS_PATH = Path(__file__).with_name("results.json")
45
46
47 # =====
48 # Problem container
49 # =====
50
51 @dataclass
52 class Portfolio:
```

```

53     """Data that defines a mean-variance portfolio problem."""
54     mu: np.ndarray          # expected returns, shape (n,)
55     Sigma: np.ndarray       # covariance matrix, shape (n, n)
56     r_min: float           # target return on the same horizon as mu
57     names: Sequence[str]   # asset names, used only for printing
58
59     @property
60     def n(self) -> int:
61         return len(self.mu)
62
63
64 def objective(p: Portfolio, w: np.ndarray) -> float:
65     """Half-variance objective  $f(w) = (1/2) w^T \Sigma w$ ."""
66     return 0.5 * w @ p.Sigma @ w
67
68
69 # =====
70 # Method 1: Logarithmic barrier method
71 # =====
72
73 def _barrier_value(p, w, t):
74     s = p.mu @ w - p.r_min
75     if np.any(w <= 0) or s <= 0:
76         return np.inf
77     return t * 0.5 * w @ p.Sigma @ w - np.sum(np.log(w)) - np.log(s)
78
79
80 def _barrier_grad(p, w, t):
81     s = p.mu @ w - p.r_min
82     return t * p.Sigma @ w - 1.0 / w - p.mu / s
83
84
85 def _barrier_hess(p, w, t):
86     s = p.mu @ w - p.r_min
87     return t * p.Sigma + np.diag(1.0 / w**2) + np.outer(p.mu, p.mu) / s**2
88
89
90 def _newton_step(p, w, t):
91     """Newton direction for the barrier subproblem with equality  $1^T w = 1$ ."""
92     n = p.n
93     g = _barrier_grad(p, w, t)
94     H = _barrier_hess(p, w, t)
95     ones = np.ones(n)
96     kkt = np.zeros((n + 1, n + 1))
97     kkt[:n, :n] = H
98     kkt[:n, n] = ones
99     kkt[n, :n] = ones
100     rhs = np.zeros(n + 1)
101     rhs[:n] = -g
102     sol = np.linalg.solve(kkt, rhs)
103     return sol[:n]
104
105
106 def barrier_method(p: Portfolio, w0, t0=1.0, gamma=10.0, eps=1e-8,
107                   newton_tol=1e-10, max_outer=100, max_newton=50):
108     """Solve the portfolio QP with a logarithmic barrier method.
109
110     Each centering step uses a damped Newton update with a backtracking line
111     search that preserves strict feasibility and enforces sufficient decrease
112     of the barrier objective.
113     """
114     w = w0.copy()
115     t = t0
116     outer = 0
117     total_newton = 0

```

```

118     log = []
119
120     while True:
121         gap = (p.n + 1) / t
122         steps = 0
123
124         for _ in range(max_newton):
125             dw = _newton_step(p, w, t)
126
127             step = 1.0
128             phi_w = _barrier_value(p, w, t)
129             slope = _barrier_grad(p, w, t) @ dw
130             for _ in range(60):
131                 w_try = w + step * dw
132                 s_try = p.mu @ w_try - p.r_min
133                 if np.all(w_try > 0) and s_try > 0:
134                     if _barrier_value(p, w_try, t) <= phi_w + 0.01 * step * slope:
135                         break
136                     step *= 0.5
137
138             w = w + step * dw
139             steps += 1
140             if np.linalg.norm(step * dw) < newton_tol:
141                 break
142
143             total_newton += steps
144             log.append({"outer": outer, "t": t, "newton": steps,
145                       "obj": objective(p, w), "gap": gap})
146
147             if gap < eps:
148                 break
149             t *= gamma
150             outer += 1
151             if outer >= max_outer:
152                 break
153
154     return w, log, total_newton
155
156
157 # =====
158 # Method 2: Primal-Dual Interior-Point Method
159 # =====
160 #
161 # We work with primal variables w and dual variables (nu, eta, lambda) at
162 # once and take a single Newton step on the relaxed KKT system at every
163 # iteration. The relaxed complementarity conditions are
164 #     eta * (mu^T w - r_min) = tau,
165 #     w_i * lambda_i = tau,      for i = 1, ..., n,
166 # where tau > 0 is driven to zero. The duality measure d_k is defined below.
167
168 def primal_dual_ipm(p: Portfolio, w0, sigma=0.1, tol=1e-10,
169                   max_iter=100, safety=0.995):
170     """Path-following primal-dual interior-point method for the portfolio QP."""
171     n = p.n
172     w = w0.copy()
173     nu = 0.0
174     eta = 1.0
175     lam = np.ones(n)
176     log = []
177
178     for k in range(max_iter):
179         s_ret = p.mu @ w - p.r_min
180         d_k = (eta * s_ret + w @ lam) / (n + 1)
181         log.append({"iter": k, "d_k": d_k, "obj": objective(p, w)})
182         if d_k < tol:

```

```

183         break
184
185     tau = sigma * d_k
186
187     r_dual = p.Sigma @ w + nu - eta * p.mu - lam
188     r_eq = np.sum(w) - 1.0
189     r_ret = eta * s_ret - tau
190     r_comp = w * lam - tau
191
192     size = 2 * n + 2
193     J = np.zeros((size, size))
194     J[:n, :n] = p.Sigma
195     J[:n, n] = 1.0
196     J[:n, n + 1] = -p.mu
197     J[:n, n + 2:] = -np.eye(n)
198     J[n, :n] = 1.0
199     J[n + 1, :n] = eta * p.mu
200     J[n + 1, n + 1] = s_ret
201     J[n + 2:, :n] = np.diag(lam)
202     J[n + 2:, n + 2:] = np.diag(w)
203
204     rhs = -np.concatenate([r_dual, [r_eq, r_ret], r_comp])
205     delta = np.linalg.solve(J, rhs)
206
207     dw = delta[:n]
208     dnu = delta[n]
209     deta = delta[n + 1]
210     dlam = delta[n + 2:]
211
212     # Fraction-to-boundary rule keeps w, lambda, eta, and the return
213     # slack strictly positive at every iteration.
214     alpha = 1.0
215     for val, step in [(w, dw), (lam, dlam)]:
216         neg = step < 0
217         if np.any(neg):
218             alpha = min(alpha, safety * np.min(-val[neg] / step[neg]))
219     if deta < 0:
220         alpha = min(alpha, safety * (-eta / deta))
221     ds_ret = p.mu @ dw
222     if ds_ret < 0:
223         alpha = min(alpha, safety * (-s_ret / ds_ret))
224
225     w = w + alpha * dw
226     nu = nu + alpha * dnu
227     eta = eta + alpha * deta
228     lam = lam + alpha * dlam
229
230     return w, log
231
232
233 # =====
234 # Method 3: Projected gradient descent
235 # =====
236
237 def _project(p: Portfolio, v):
238     """Euclidean projection of v onto the feasible set, computed by SLSQP."""
239     res = sp_minimize(
240         lambda w: 0.5 * np.sum((w - v) ** 2),
241         x0=np.ones(p.n) / p.n,
242         jac=lambda w: w - v,
243         method="SLSQP",
244         constraints=[
245             {"type": "eq", "fun": lambda w: np.sum(w) - 1},
246             {"type": "ineq", "fun": lambda w: p.mu @ w - p.r_min},
247         ],

```



```

248         bounds=[(0, None)] * p.n,
249         options={"ftol": 1e-15, "maxiter": 1000},
250     )
251     return res.x
252
253
254 def projected_gradient(p: Portfolio, w0, tol=1e-8, max_iter=10000):
255     """Projected gradient descent with step size 1 / lambda_max(Sigma)."""
256     eigvals = np.linalg.eigvalsh(p.Sigma)
257     alpha = 1.0 / eigvals[-1]
258
259     w = w0.copy()
260     iters = 0
261     for _ in range(max_iter):
262         grad = p.Sigma @ w
263         w_new = _project(p, w - alpha * grad)
264         iters += 1
265         if np.linalg.norm(w_new - w) < tol:
266             w = w_new
267             break
268         w = w_new
269     return w, iters
270
271
272 # =====
273 # Runner
274 # =====
275
276 def time_it(fn: Callable, *args, trials: int = 5, **kwargs):
277     """Run a function several times and return (result, median_seconds).
278
279     Each method is deterministic, so iteration counts and final iterates do
280     not change between trials. Only wall-clock time varies; reporting the
281     median reduces timing noise.
282     """
283     times = []
284     out = None
285     for _ in range(trials):
286         t0 = time.perf_counter()
287         out = fn(*args, **kwargs)
288         times.append(time.perf_counter() - t0)
289     return out, float(np.median(times))
290
291
292 def run_all(p: Portfolio, label: str):
293     """Run all three methods on the same problem and return a summary dict."""
294     print("\n" + "=" * 72)
295     print(f"EXPERIMENT: {label}")
296     print("=" * 72)
297
298     eigvals = np.linalg.eigvalsh(p.Sigma)
299     cond = float(eigvals[-1] / eigvals[0])
300     w_unif = np.ones(p.n) / p.n
301
302     print(f"Assets           : {list(p.names)}")
303     print(f"Expected returns    : {np.round(p.mu, 4)}")
304     print(f"Condition number     : {cond:.2f}")
305     print(f"Target return       : {p.r_min:.4f}")
306     print(f"Uniform return       : {p.mu @ w_unif:.4f}")
307     print(f"Slater satisfied     : {bool(p.mu @ w_unif > p.r_min)}")
308
309     w0 = np.ones(p.n) / p.n
310
311     (w_bar, _, bar_newton), t_bar = time_it(
312         barrier_method, p, w0, t0=1.0, gamma=10.0, eps=1e-8

```

```

313 )
314 (w_pd, pd_log), t_pd = time_it(
315     primal_dual_ipm, p, w0, sigma=0.1, tol=1e-10
316 )
317 (w_pg, pg_iters), t_pg = time_it(
318     projected_gradient, p, w0, tol=1e-8, max_iter=10000
319 )
320
321 print("\nMethod                ITERS    Time (s)    Objective")
322 print("-" * 68)
323 print(f"Projected gradient descent    {pg_iters:>5}    "
324       f"{t_pg:>8.4f}    {objective(p, w_pg):.6e}")
325 print(f"Barrier method                {bar_newton:>5}    "
326       f"{t_bar:>8.4f}    {objective(p, w_bar):.6e}")
327 print(f"Primal-dual IPM                {len(pd_log) - 1:>5}    "
328       f"{t_pd:>8.4f}    {objective(p, w_pd):.6e}")
329
330 print("\nWeights (barrier)                : "
331       f"{np.array2string(w_bar, precision=4)}")
332 print(f"||w_pd - w_bar||                : {np.linalg.norm(w_pd - w_bar):.2e}")
333 print(f"||w_pg - w_bar||                : {np.linalg.norm(w_pg - w_bar):.2e}")
334
335 return {
336     "label": label,
337     "cond_Sigma": cond,
338     "mu": p.mu.tolist(),
339     "r_min": p.r_min,
340     "names": list(p.names),
341     "barrier": {
342         "iters": int(bar_newton),
343         "time_s": t_bar,
344         "obj": objective(p, w_bar),
345         "w": w_bar.tolist(),
346     },
347     "primal_dual": {
348         "iters": len(pd_log) - 1,
349         "time_s": t_pd,
350         "obj": objective(p, w_pd),
351         "w": w_pd.tolist(),
352         "dist_to_barrier": float(np.linalg.norm(w_pd - w_bar)),
353     },
354     "projected_gradient": {
355         "iters": int(pg_iters),
356         "time_s": t_pg,
357         "obj": objective(p, w_pg),
358         "w": w_pg.tolist(),
359         "dist_to_barrier": float(np.linalg.norm(w_pg - w_bar)),
360     },
361 }
362
363
364 # =====
365 # Experiment A: simulated data
366 # =====
367
368 def make_simulated() -> Portfolio:
369     """Return the simulated 5-asset problem used in Section 5.1.
370
371     The legacy NumPy RandomState API is used so that the seed reproduces the
372     exact covariance matrix reported in the project (condition number 13.6).
373     """
374     np.random.seed(42)
375     n = 5
376     mu = np.array([0.12, 0.10, 0.07, 0.03, 0.05])
377     A = np.random.randn(n, n)

```

```

378     Sigma = A.T @ A / 100 + 0.01 * np.eye(n)
379     return Portfolio(mu=mu, Sigma=Sigma, r_min=0.06,
380                     names=[f"A{i+1}" for i in range(n)])
381
382
383 # =====
384 # Experiment B: real market data
385 # =====
386
387 def _download_prices(tickers, start, end):
388     """Download daily adjusted closing prices over a fixed [start, end] window.
389
390     The yfinance import is performed inside the function so that the
391     simulated experiment can run without yfinance installed. The optional
392     curl_cffi session is used when available to avoid rate limiting.
393     """
394     try:
395         import yfinance as yf
396     except ImportError as exc:
397         raise ImportError(
398             "The real-data experiment requires the 'yfinance' package. "
399             "Install it with 'pip install yfinance' (and optionally "
400             "'pip install curl_cffi' to avoid Yahoo rate limiting)."
401         ) from exc
402
403     try:
404         from curl_cffi import requests as cr
405         session = cr.Session(impersonate="chrome")
406     except ImportError:
407         session = None
408
409     raw = yf.download(
410         tickers,
411         start=start,
412         end=end,
413         auto_adjust=True,
414         progress=False,
415         threads=False,
416         session=session,
417     )
418     closes = raw["Close"] if "Close" in raw.columns.levels[0] else raw
419     return closes[list(tickers)].dropna()
420
421
422 def make_real(tickers, start=REAL_DATA_START, end=REAL_DATA_END,
423              r_min_frac=0.5):
424     """Build an annualized mean-variance problem from real daily prices.
425
426     Daily simple returns are scaled by TRADING_DAYS_PER_YEAR so that mu and
427     Sigma are on the same annualized horizon.
428     """
429     prices = _download_prices(tickers, start, end)
430     daily = prices.pct_change().dropna()
431     mu = daily.mean().to_numpy() * TRADING_DAYS_PER_YEAR
432     Sigma = daily.cov().to_numpy() * TRADING_DAYS_PER_YEAR
433
434     r_min = r_min_frac * float(np.max(mu))
435     return Portfolio(mu=mu, Sigma=Sigma, r_min=r_min,
436                     names=list(tickers), prices.index)
437
438
439 # =====
440 # Entry point
441 # =====
442

```

```

443 def main():
444     print("Interior-point methods: simulated and real-data experiments.")
445     print("Hardware: MacBook Air 13-inch, M3, 2024.\n")
446
447     sim_problem = make_simulated()
448     sim_result = run_all(sim_problem, "Simulated 5-asset problem")
449
450     tickers = ("AAPL", "GOOGL", "META", "AMZN", "MSFT")
451     real_problem, dates = make_real(tickers,
452                                     start=REAL_DATA_START,
453                                     end=REAL_DATA_END,
454                                     r_min_frac=0.5)
455     print(f"\nReal data window : {dates.min().date()} to {dates.max().date()}")
456     real_result = run_all(
457         real_problem,
458         f"Real data: {' ', ' '.join(tickers)} ({REAL_DATA_START} to {REAL_DATA_END})",
459     )
460     real_result["window"] = {
461         "start": str(dates.min().date()),
462         "end": str(dates.max().date()),
463     }
464
465     payload = {
466         "hardware": "MacBook Air 13-inch, M3, 2024",
467         "trading_days_per_year": TRADING_DAYS_PER_YEAR,
468         "simulated": sim_result,
469         "real": real_result,
470     }
471     RESULTS_PATH.write_text(json.dumps(payload, indent=2))
472     print(f"\nResults written to {RESULTS_PATH.name}")
473
474
475 if __name__ == "__main__":
476     main()

```

References

- [1] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
<https://www.jstor.org/stable/2975974>
- [2] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
<https://web.stanford.edu/~boyd/cvxbook/>
- [3] S. J. Wright, *Primal-Dual Interior-Point Methods*, SIAM, 1997.
<https://doi.org/10.1137/1.9781611971453>
- [4] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
<https://doi.org/10.1007/978-0-387-40065-5>
- [5] M. S. Bazaraa, H. D. Sherali, and C. M. Shetty, *Nonlinear Programming: Theory and Algorithms*, 3rd ed., Wiley, 2006.
<https://doi.org/10.1002/0471787779>
- [6] M. K. Riahi, *Nonlinear Optimization: Theory and Practice*, Lecture Notes, Math 707, Springer 2026.